Guía del alumno

Módulo 3, Limpieza y análisis exploratorio

Ruta guiada

Análisis Avanzado de Datos con Python

Cómo usar esta guía

Esta guía acompaña al Lab 03 guiado y sigue sus dos partes en el mismo orden y con los mismos nombres del cuaderno. La Parte 1, En clase, tiene seis pasos, de Paso 1 de 6 a Paso 6 de 6, y se hace en clase. La Parte 2, Para estudiar, tiene tres estudios, de Estudio 1 de 3 a Estudio 3 de 3, y se hace después de clase con esta guía, una vez corrida la Parte 1. Cada paso trae el código y la respuesta tal como aparecen en Colab, y una explicación en palabras simples.

En los pasos no hay que escribir código. Lo que cambia se cambia en los deslizadores y en la lista desplegable, y después se vuelve a correr la celda. El lab responde tres preguntas. Qué hay que mirar en un archivo antes de calcularle nada, qué hacer con las filas repetidas, los datos que faltan y los valores imposibles, y cuánto cambia el resultado según cómo se limpie.

Lo que vas a poder hacer al terminar

- Contar las filas de un archivo y compararlas con las que debería traer.
- Leer bien un número con coma decimal y una fecha que empieza por el día.
- Unificar las formas distintas de escribir una misma región.
- Quitar filas repetidas y decidir qué hacer con los datos que faltan.
- Encontrar valores imposibles con una regla de negocio.
- Saber cuándo el promedio no describe bien a un grupo.

Todas las respuestas de esta guía vienen de una corrida del cuaderno con los valores que trae por defecto. Los datos del curso son sintéticos, inventados para aprender, y no se citan como datos del sector.

Parte 1 · En clase

Seis pasos, uno detrás del otro, que se hacen en clase en el mismo orden del cuaderno.

Paso 1 de 6 · ¿Cómo llegó el archivo?

Antes de calcular nada se mira cómo llegó el archivo. Este es un archivo de demanda de cada hora de 2024 en las siete regiones, tal como llegaría desde una planilla, con errores.

Paso 1 de 6

```
# Paso 1
DATOS = "https://raw.githubusercontent.com/RodrigoMSB/curso-python-minergia/main/datos/"
import pandas as pd

# El archivo viene separado por punto y coma, como lo guarda Excel en Chile.
sucia = pd.read_csv(DATOS + "demanda_sucia.csv", sep=";")

# 2024 fue bisiesto. 366 días por 24 horas por 7 regiones.
esperadas = 366 * 24 * 7
print("Filas del archivo ", len(sucia))
print("Filas esperadas   ", esperadas)
print("Sobran            ", len(sucia) - esperadas)
sucia.head()
```

```
Filas del archivo  61508
Filas esperadas    61488
Sobran             20
        fecha  hora       region    mwh
0  01/01/2024     0  Antofagasta  51,95
1  01/01/2024     1  Antofagasta  49,02
2  01/01/2024     2  Antofagasta  48,22
3  01/01/2024     3  Antofagasta  45,57
4  01/01/2024     4  Antofagasta  47,59
```

El archivo viene separado por punto y coma en vez de coma, como lo guarda Excel en Chile. Por eso la lectura lleva `sep=";"`.

La primera pregunta es si trae las filas que debería. 2024 fue bisiesto, así que son 366 días por 24 horas por 7 regiones, 61488 filas. El archivo trae 61508, sobran 20. Todavía no sabemos por qué, eso lo responde el Paso 4.

Mirando las primeras filas ya se ve algo raro. La demanda aparece con coma decimal, `51,95`, y la fecha con el día primero, `01/01/2024`.

Paso 2 de 6 · ¿Cómo se leen bien el número y la fecha?

Python usa el punto para los decimales y escribe las fechas con el año primero. Este archivo trae coma decimal y la fecha con el día primero. La celda lo vuelve a leer y le avisa a pandas cómo viene escrito.

Paso 2 de 6

```
# Paso 2
# Se lee el mismo archivo, ahora avisando cómo viene escrito.
# decimal="," avisa que la coma separa los decimales.
# dayfirst=True avisa que en la fecha va primero el día, como en Chile.
limpia = pd.read_csv(DATOS + "demanda_sucia.csv", sep=";", decimal=",",
                     parse_dates=["fecha"], dayfirst=True)

print("Demanda total del archivo", round(limpia["mwh"].sum()), "MWh")
print("Primer día", limpia["fecha"].min().date())
print("Último día", limpia["fecha"].max().date())
limpia.head()
```

```
Demanda total del archivo 13560560 MWh
Primer día 2024-01-01
Último día 2024-12-31
       fecha  hora       region    mwh
0 2024-01-01     0  Antofagasta  51.95
1 2024-01-01     1  Antofagasta  49.02
2 2024-01-01     2  Antofagasta  48.22
3 2024-01-01     3  Antofagasta  45.57
4 2024-01-01     4  Antofagasta  47.59
```

La lectura es la misma del Paso 1, con tres avisos más. `decimal=","` dice que la coma separa los decimales, así `51,95` se lee como el número 51.95. `parse_dates` dice que la columna `fecha` es una fecha, y `dayfirst=True` que en esa fecha va primero el día.

La tabla nueva se llama `limpia`. En sus primeras filas la demanda ya aparece con punto, 51.95, y la fecha como 2024-01-01, que es como Python muestra una fecha. Como la demanda ya es un número, se puede sumar. Todo el archivo suma 13560560 MWh y las fechas van del 1 de enero al 31 de diciembre de 2024.

La tabla `sucia` no cambia. Queda tal como llegó el archivo, para compararla en los pasos que siguen.

Paso 3 de 6 · ¿De cuántas formas viene escrita cada región?

Una misma región puede venir escrita de varias formas, con espacios de más o con mayúsculas distintas. Para Python, Biobío, biobío y BIOBÍO son tres regiones distintas.

Paso 3 de 6

```
# Paso 3
# nunique cuenta cuántos valores distintos tiene una columna.
print("Formas distintas en el archivo   ", sucia["region"].nunique())

# strip quita los espacios del comienzo y del final.
limpia["region"] = limpia["region"].str.strip()
print("Después de quitar los espacios   ", limpia["region"].nunique())

# title deja la primera letra de cada palabra en mayúscula y el resto en minúscula.
limpia["region"] = limpia["region"].str.title()
print("Después de igualar las mayúsculas", limpia["region"].nunique())
```

```
Formas distintas en el archivo    49
Después de quitar los espacios    21
Después de igualar las mayúsculas 7
```

`nunique` cuenta cuántos valores distintos tiene una columna. En el archivo tal como llegó, la región viene escrita de 49 formas, cuando las regiones son 7.

La celda arregla la columna `region` de `limpia` en dos líneas. `str.strip` quita los espacios del comienzo y del final, y las formas bajan a 21. `str.title` deja la primera letra de cada palabra en mayúscula y el resto en minúscula, y quedan las 7 regiones.

Si la celda se corre dos veces, la segunda vez la línea del medio también dice 7, porque la columna ya quedó arreglada la primera vez. Para ver de nuevo 49, 21 y 7 hay que correr el Paso 2 y después el Paso 3.

Paso 4 de 6 · ¿Hay filas repetidas?

En el Paso 1 sobraban 20 filas. Esta celda revisa si son copias.

Paso 4 de 6

```
# Paso 4
# duplicated marca cada fila idéntica a una que ya apareció antes.
print("Filas repetidas ", limpia.duplicated().sum())

# drop_duplicates deja la primera y quita las copias.
sin_repetidas = limpia.drop_duplicates()
print("Filas que quedan", len(sin_repetidas))
print("Filas esperadas ", esperadas)
```

```
Filas repetidas  20
Filas que quedan 61488
Filas esperadas  61488
```

`duplicated` marca cada fila idéntica a otra que ya apareció antes. Hay 20, justo las que sobraban en el Paso 1. `drop_duplicates` deja la primera de cada fila repetida y quita las copias. Quedan 61488 filas, las mismas que se esperaban.

Las filas repetidas se quitan antes de sumar. Si no, la demanda de esas horas se cuenta dos veces y el total queda más alto de lo que fue. La suma del Paso 2 todavía contaba esas copias.

Paso 5 de 6 · ¿Con qué se rellena una hora vacía?

Quedan las horas que vinieron sin dato. Hay varias formas de rellenarlas y cada una cambia el resultado. Para verlo se usa una sola región, Valparaíso.

Paso 5 de 6

```
# Paso 5
# Todas las horas de Valparaíso, en orden. copy la deja como una tabla aparte.
valpo = sin_repetidas[sin_repetidas["region"] == "Valparaíso"].copy()
print("Horas sin dato", valpo["mwh"].isna().sum())

# Una columna nueva por cada forma de rellenar las horas vacías.
valpo["poner_cero"] = valpo["mwh"].fillna(0)
valpo["repetir_anterior"] = valpo["mwh"].ffill()
valpo["trazar_recta"] = valpo["mwh"].interpolate().round(2)

# El promedio por hora del año con cada forma. mean salta las horas vacías.
print("Promedio borrando las vacías", round(valpo["mwh"].mean(), 1))
print("Promedio poniendo cero      ", round(valpo["poner_cero"].mean(), 1))
print("Promedio repitiendo         ", round(valpo["repetir_anterior"].mean(), 1))
print("Promedio con una recta      ", round(valpo["trazar_recta"].mean(), 1))

# Un día que vino con ocho horas vacías.
dia = valpo[valpo["fecha"] == "2024-09-16"]
dia
```

```
Horas sin dato 248
Promedio borrando las vacías 208.9
Promedio poniendo cero       203.0
Promedio repitiendo          208.8
Promedio con una recta       208.8
           fecha  hora      region     mwh  poner_cero  repetir_anterior  \
32579 2024-09-16     0  Valparaíso  177.03      177.03             177.03
32580 2024-09-16     1  Valparaíso     NaN        0.00             177.03
32581 2024-09-16     2  Valparaíso     NaN        0.00             177.03
32582 2024-09-16     3  Valparaíso  165.98      165.98             165.98
32583 2024-09-16     4  Valparaíso  164.67      164.67             164.67
32584 2024-09-16     5  Valparaíso  179.25      179.25             179.25
32585 2024-09-16     6  Valparaíso  191.42      191.42             191.42
32586 2024-09-16     7  Valparaíso  222.46      222.46             222.46
32587 2024-09-16     8  Valparaíso     NaN        0.00             222.46
32588 2024-09-16     9  Valparaíso     NaN        0.00             222.46
32589 2024-09-16    10  Valparaíso  247.39      247.39             247.39
32590 2024-09-16    11  Valparaíso  259.55      259.55             259.55
32591 2024-09-16    12  Valparaíso  257.45      257.45             257.45
32592 2024-09-16    13  Valparaíso  252.23      252.23             252.23
32593 2024-09-16    14  Valparaíso  240.87      240.87             240.87
32594 2024-09-16    15  Valparaíso     NaN        0.00             240.87
32595 2024-09-16    16  Valparaíso     NaN        0.00             240.87
32596 2024-09-16    17  Valparaíso  251.28      251.28             251.28
32597 2024-09-16    18  Valparaíso  263.82      263.82             263.82
32598 2024-09-16    19  Valparaíso  273.52      273.52             273.52
32599 2024-09-16    20  Valparaíso  276.05      276.05             276.05
32600 2024-09-16    21  Valparaíso  254.03      254.03             254.03
32601 2024-09-16    22  Valparaíso     NaN        0.00             254.03
32602 2024-09-16    23  Valparaíso     NaN        0.00             254.03

       trazar_recta
32579        177.03
32580        173.35
32581        169.66
32582        165.98
32583        164.67
32584        179.25
32585        191.42
32586        222.46
32587        230.77
32588        239.08
32589        247.39
32590        259.55
32591        257.45
32592        252.23
32593        240.87
32594        244.34
32595        247.81
32596        251.28
32597        263.82
32598        273.52
32599        276.05
32600        254.03
32601        230.89
32602        207.76
```

La celda deja las horas de Valparaíso en una tabla aparte, `valpo`. Tiene 248 horas sin dato.

Después agrega una columna por cada forma de rellenar. `fillna(0)` pone cero en cada hora vacía. `ffill` repite el valor de la hora anterior. `interpolate` traza una recta entre la hora anterior y la siguiente, y `round(2)` la deja con dos decimales. La cuarta forma es no rellenar nada, y es lo que hace `mean`, que salta las horas vacías.

Los promedios cambian según la forma. Borrando las vacías el promedio por hora queda en 208.9 MWh, poniendo cero baja a 203.0, y repitiendo o con una recta queda en 208.8.

La tabla es el 16 de septiembre, un día que vino con ocho horas vacías. A la 1 y a las 2 de la mañana

`mwh` viene vacío y aparece `NaN`. Poner cero deja 0.00, repetir deja 177.03 en las dos horas y la recta baja de a poco, 173.35 y 169.66, entre sus vecinas.

La tabla es más ancha que la hoja, así que en esta guía las últimas columnas aparecen más abajo. En Colab se ve completa.

> **LA DECISIÓN QUE NO SE VE EN EL INFORME**
>
> Con los mismos datos, cada forma de rellenar deja un resultado distinto. Poner cero baja el promedio y repetir el valor anterior aplana el día. Ninguna es la correcta siempre, pero la que se use se deja escrita.

Paso 6 de 6 · Tu turno, decide qué valor es imposible

Algunas horas traen un valor que no puede ser real. Se encuentran con reglas de negocio, que son reglas que salen de conocer el tema.

Paso 6 de 6

```
# Paso 6
region = "Atacama" #@param ["Antofagasta", "Atacama", "Coquimbo", "Valparaíso", "Metropolitana",
↪  "Biobío", "Los Lagos"]
veces = 3 #@param {type:"slider", min:1, max:10, step:1}

una = sin_repetidas[sin_repetidas["region"] == region]

# describe resume la columna en ocho números. Mira el mínimo y el máximo.
print(una["mwh"].describe().round(1))

# Dos reglas de negocio. La demanda no puede ser negativa,
# y una hora muchas veces mayor que el promedio es sospechosa.
promedio = una["mwh"].mean()
negativas = una[una["mwh"] < 0]
picos = una[una["mwh"] > veces * promedio]
print("Horas con demanda negativa", len(negativas))
print(f"Horas sobre {veces} veces el promedio", len(picos))
```

```
count    8512.0
mean       35.2
std        11.3
min       -42.6
25%        29.7
50%        34.5
75%        41.0
max       442.7
Name: mwh, dtype: float64
Horas con demanda negativa 5
Horas sobre 3 veces el promedio 6
```

La celda toma las horas de la región que se elige en la lista, Atacama si no se cambia. `describe` resume la columna `mwh` en ocho números. Son las horas con dato, el promedio, la desviación estándar, el mínimo, los valores bajo los que queda el 25, el 50 y el 75 por ciento de las horas, y el máximo. En Atacama el mínimo es -42.6, una demanda negativa que no puede ser real. El máximo es 442.7, más de diez veces el promedio de 35.2, y eso es muy sospechoso para una sola hora.

Después la celda aplica dos reglas de negocio. La demanda no puede ser negativa, y hay 5 horas así. Y una hora muchas veces mayor que el promedio es sospechosa. Con el deslizador en 3 hay 6 horas sobre tres veces el promedio.

Mueve el deslizador a 1 y vuelve a correr la celda. Marca 4000 horas, casi la mitad del año, porque muchas horas normales superan el promedio. Con 10 marca solo 2, y se escapan picos. Elige Metropolitana en la lista, con el deslizador en 3. Su máximo es 11195.0, con 1 hora negativa y 7 sobre tres veces el promedio.

EL UMBRAL LO DECIDE QUIEN ANALIZA

Con 1 vez el promedio marca casi la mitad de las horas, y son horas normales. Entre 2 y 7 veces marca siempre los mismos 6 picos de Atacama, los que se plantaron al crear los datos. Con 10 se escapan picos. En el trabajo real los valores imposibles se vacían y se rellenan como en el Paso 5, y el umbral que se usó se deja escrito.

Parte 2 · Para estudiar

Estos pasos se hacen después de clase, con esta guía. Antes de empezar hay que correr toda la Parte 1 del cuaderno, porque los estudios usan sus tablas.

Estudio 1 de 3 · ¿Qué forma de rellenar se acercó más a la verdad?

Este curso tiene algo que el trabajo real no tiene, el archivo original sin errores. Con él se puede medir cuánto se equivocó cada forma de rellenar del Paso 5.

Estudio 1 de 3

```
# Estudio 1
# Este curso tiene algo que el trabajo real no tiene, el archivo original sin errores.
verdad = pd.read_csv(DATOS + "demanda.csv", parse_dates=["fecha"])
real = verdad[(verdad["region"] == "Valparaíso") & (verdad["fecha"] == "2024-09-16")]

# Se copia el día del Paso 5 y se le agrega la demanda real de cada hora.
# list toma los 24 valores de real, en orden.
comparar = dia.copy()
comparar["real"] = list(real["mwh"])
comparar
```

```
          fecha  hora      region     mwh  poner_cero  repetir_anterior  \
32579 2024-09-16     0  Valparaíso  177.03      177.03            177.03
32580 2024-09-16     1  Valparaíso     NaN        0.00            177.03
32581 2024-09-16     2  Valparaíso     NaN        0.00            177.03
32582 2024-09-16     3  Valparaíso  165.98      165.98            165.98
32583 2024-09-16     4  Valparaíso  164.67      164.67            164.67
32584 2024-09-16     5  Valparaíso  179.25      179.25            179.25
32585 2024-09-16     6  Valparaíso  191.42      191.42            191.42
32586 2024-09-16     7  Valparaíso  222.46      222.46            222.46
32587 2024-09-16     8  Valparaíso     NaN        0.00            222.46
32588 2024-09-16     9  Valparaíso     NaN        0.00            222.46
32589 2024-09-16    10  Valparaíso  247.39      247.39            247.39
32590 2024-09-16    11  Valparaíso  259.55      259.55            259.55
32591 2024-09-16    12  Valparaíso  257.45      257.45            257.45
32592 2024-09-16    13  Valparaíso  252.23      252.23            252.23
32593 2024-09-16    14  Valparaíso  240.87      240.87            240.87
32594 2024-09-16    15  Valparaíso     NaN        0.00            240.87
32595 2024-09-16    16  Valparaíso     NaN        0.00            240.87
32596 2024-09-16    17  Valparaíso  251.28      251.28            251.28
32597 2024-09-16    18  Valparaíso  263.82      263.82            263.82
32598 2024-09-16    19  Valparaíso  273.52      273.52            273.52
32599 2024-09-16    20  Valparaíso  276.05      276.05            276.05
32600 2024-09-16    21  Valparaíso  254.03      254.03            254.03
32601 2024-09-16    22  Valparaíso     NaN        0.00            254.03
32602 2024-09-16    23  Valparaíso     NaN        0.00            254.03

       trazar_recta    real
32579        177.03  177.03
32580        173.35  168.14
32581        169.66  167.37
32582        165.98  165.98
32583        164.67  164.67
32584        179.25  179.25
32585        191.42  191.42
32586        222.46  222.46
32587        230.77  234.15
32588        239.08  241.76
32589        247.39  247.39
32590        259.55  259.55
32591        257.45  257.45
32592        252.23  252.23
32593        240.87  240.87
32594        244.34  246.52
32595        247.81  250.76
32596        251.28  251.28
32597        263.82  263.82
32598        273.52  273.52
32599        276.05  276.05
32600        254.03  254.03
32601        230.89  220.11
32602        207.76  198.94
```

La celda lee el archivo original, `demanda.csv`, y se queda con el mismo día de Valparaíso del Paso

5. Después copia la tabla `dia` y le agrega la columna `real`, con la demanda verdadera de cada hora. `list` toma los 24 valores en orden, así cada uno queda en su hora.

En las horas que traían dato las columnas coinciden. Las diferencias están en las ocho horas que vinieron vacías. A la 1 de la mañana la demanda real fue 168.14. Poner cero dejó 0.00, repetir dejó 177.03 y la recta 173.35, la más cercana. A las 9 la real fue 241.76, repetir dejó 222.46 y la recta 239.08.

La tabla es más ancha que la hoja, así que en esta guía las últimas columnas aparecen más abajo. En Colab se ve completa.

> **ESTO NO SE PUEDE HACER EN EL TRABAJO REAL**
>
> Aquí se pudo medir el error porque existe el archivo original. En las ocho horas vacías, poner cero se equivoca en unos 200 MWh por hora, repetir el valor anterior hasta en 55 y la recta entre 2 y 11. En el trabajo real el dato sucio es el único que hay. Por eso la forma de rellenar se elige con criterio y se deja escrita.

Estudio 2 de 3 · Tu turno, mira cómo está escrita cada región

Las 49 formas de escribir las regiones, contadas de la más frecuente a la menos frecuente.

Estudio 2 de 3

```
# Estudio 2
cuantas = 12 #@param {type:"slider", min:5, max:49, step:1}

# value_counts cuenta cuántas veces aparece cada forma, de mayor a menor.
sucia["region"].value_counts().head(cuantas)
```

```
region
Valparaíso       8714
Antofagasta      8713
Coquimbo         8707
Metropolitana    8698
Atacama          8693
Los Lagos        8686
Biobío           8683
biobío             26
los lagos          21
BIOBÍO             21
metropolitana      20
atacama            19
Name: count, dtype: int64
```

`value_counts` cuenta cuántas veces aparece cada forma. Las siete primeras son las regiones bien escritas, con más de 8000 horas cada una. Desde la octava aparecen las mal escritas, en minúscula, en mayúscula o con espacios. La octava es `biobío`, con 26 horas.

Mueve el deslizador para ver más filas y vuelve a correr la celda. Si una región mal escrita se queda así, cualquier resumen por región la cuenta como si fuera otra región.

Estudio 3 de 3 · ¿Cuánto consume una región en una hora normal?

Con la tabla sin filas repetidas se puede describir la demanda. La pregunta es si el promedio describe bien una hora típica.

Estudio 3 de 3

```
# Estudio 3
# La mediana es el valor del medio. La mitad de las horas está por debajo y la otra mitad por
↪  encima.
print("Todo el país junto")
print("  Promedio", round(sin_repetidas["mwh"].mean(), 1), " Mediana",
↪  round(sin_repetidas["mwh"].median(), 1))

resumen = sin_repetidas.groupby("region")["mwh"].agg(promedio="mean", mediana="median")
resumen.round(1).sort_values("promedio", ascending=False)
```

```
Todo el país junto
  Promedio 227.3  Mediana 111.2
              promedio  mediana
region
Metropolitana     892.3    873.5
Valparaíso        208.9    204.8
Biobío            186.9    182.7
Los Lagos          99.1     96.9
Coquimbo           93.3     91.6
Antofagasta        76.9     75.6
Atacama            35.2     34.5
```

La **mediana** es el valor del medio. La mitad de las horas está por debajo y la otra mitad por encima. Con todas las regiones juntas, el promedio es 227.3 MWh y la mediana 111.2, menos de la mitad. Pero dentro de cada región los dos números casi coinciden. En la Metropolitana el promedio es 892.3 y la mediana 873.5. En Atacama, 35.2 y 34.5.

> **EL PROMEDIO DE TODO EL PAÍS NO DESCRIBE A NINGUNA REGIÓN**
>
> Con todas las regiones juntas el promedio es el doble de la mediana. No es por los valores imposibles, que son pocos y casi no lo mueven. Es porque la Metropolitana consume varias veces más que las demás. Dentro de cada región el promedio y la mediana están casi iguales.

Para seguir practicando

Al final del cuaderno hay cuatro ejercicios para quien quiera seguir después de clase. En ellos sí se escribe código. Antes de empezar hay que correr todo el cuaderno con Entorno de ejecución, Ejecutar todo, porque los ejercicios usan las tablas de los pasos.

Cada ejercicio tiene una celda vacía para escribir la respuesta y una solución plegada, que conviene abrir solo después de intentarlo. Las soluciones no están en esta guía.

- **Ejercicio 1.** Arreglar el número y la fecha después de leer el archivo, en vez de avisarlo al leer.
- **Ejercicio 2.** Limpiar la tabla completa, vaciando los valores imposibles de todas las regiones y rellenando con una recta.
- **Ejercicio 3.** Calcular la demanda alta de cada región con el percentil 95.
- **Ejercicio 4.** Armar una tabla con la demanda promedio por región y por franja del día.

Para repasar

Referencia rápida

Lo que se escribe	Para qué sirve
`pd.read_csv(..., sep=";")`	Lee un archivo separado por punto y coma
`decimal=","`	Avisa que la coma separa los decimales
`parse_dates=[...]` · `dayfirst=True`	Lee la fecha como fecha, con el día primero
`nunique()`	Cuántos valores distintos tiene una columna
`str.strip()` · `str.title()`	Quita espacios e iguala las mayúsculas
`duplicated()` · `drop_duplicates()`	Marca y quita filas repetidas
`fillna(0)` · `ffill()` · `interpolate()`	Tres formas de rellenar huecos
`describe()`	Resume una columna en ocho números
`value_counts()`	Cuántas veces aparece cada valor
`mean()` · `median()`	Promedio y mediana

Si algo sale mal

Lo que aparece en pantalla	Qué pasó
`NameError: name 'sucia' is not defined`	Corriste un paso sin correr antes el Paso 1, que es el que lee el archivo.
`NameError: name 'sin_repetidas' is not defined`	Los Pasos 5 y 6 y el Estudio 3 usan la tabla que arma el Paso 4. Corre el Paso 4 antes.
`NameError: name 'dia' is not defined`	El Estudio 1 usa el día que arma el Paso 5. Corre el Paso 5 antes.
En el Paso 3 la línea del medio dice 7	Corriste el Paso 3 dos veces. Corre el Paso 2 y después el Paso 3.
La respuesta no cambió después de mover el deslizador o cambiar la lista	Cambiar un campo no corre la celda. Hay que volver a apretar el botón de play.
`ValueError: could not convert string to float`	Aparece en el Ejercicio 1 si se convierte la demanda a número sin cambiar antes la coma por punto.

Tres preguntas de repaso

1. Por qué la demanda llegó como texto

Porque viene con coma decimal. Python usa el punto para los decimales, así que sin `decimal=","` lee `51,95` como texto.

2. Por qué hay que dejar escrita la forma de rellenar los huecos

Porque cada forma da un resultado distinto con los mismos datos, y quien lea el informe tiene que saber cuál se usó.

3. Por qué el umbral de lo imposible lo decide quien analiza

Porque cambia cuántas horas se marcan. Con 1 vez el promedio se marcan horas normales y con 10 se escapan picos. El umbral que se usó se deja escrito.

El Lab 03 guiado en una plana

Para imprimir y tener al lado durante el curso

Los pasos de las dos partes

N.	La pregunta	La idea
Parte 1, en clase		
1	¿Cómo llegó el archivo?	Contar las filas
2	¿Cómo se leen bien el número y la fecha?	Avisar la coma y el día primero
3	¿De cuántas formas viene escrita cada región?	Igualar los textos
4	¿Hay filas repetidas?	Quitar las copias
5	¿Con qué se rellena una hora vacía?	Cuatro formas de rellenar
6	Tu turno, decide qué valor es imposible	Reglas de negocio y umbral
Parte 2, para estudiar		
1	¿Qué forma de rellenar se acercó más a la verdad?	Comparar con el dato real
2	Tu turno, mira cómo está escrita cada región	Una región escrita de muchas formas
3	¿Cuánto consume una región en una hora normal?	Promedio y mediana

El orden de una limpieza

Primero mirar	Contar filas, revisar tipos y valores distintos
Después convertir	Números, fechas y textos a su forma correcta
Después quitar copias	Antes de sumar nada
Rellenar con criterio	Cada forma de rellenar cambia el resultado
Dejar escrito el criterio	La forma de rellenar y el umbral de lo imposible

Los métodos estadísticos para encontrar valores extremos y el detalle de cada estrategia de relleno están en la ruta completa de este módulo.